

1. What is Cloud?

The Cloud refers to a network of remote servers connected over the internet, used to store data, run applications, manage services, and host resources without needing physical hardware on your premises.

✓ Why Cloud Was Needed

Before cloud computing:

- Companies had to buy expensive servers.
- Hardware had to be maintained manually.
- If demand increased, companies needed to buy more servers.
- Scaling was slow, expensive, and difficult.
- Backups and security had to be managed manually.

✓ What Cloud Solves

Cloud providers like Microsoft Azure, AWS, and Google Cloud offer:

- On-demand resources → Use whenever needed.
- Scalability → Increase or decrease resources instantly.
- Pay only for what you use → No hardware cost.
- High availability → 99.9% uptime.
- Global presence → Deploy anywhere in the world.
- Security & compliance → Handled by cloud provider.

✓ Simple Example

If your app suddenly gets 10,000 new users today:

- On-premises: Your server might crash.
- On cloud: Azure automatically scales to handle load.

2. What is DevOps?

DevOps is not a tool, not a software, and not a single process.

It is a culture + mindset + practices + tools combination that brings Development and Operations teams together.

✓ Purpose of DevOps

Before DevOps:

- Developers wrote code.
- Operations deployed code on servers.
- Both teams worked separately.

This caused:

- Constant miscommunication
- Delays in deployment
- Bugs not found early
- Conflicts ("your code is wrong" vs "your server is wrong")
- Different environments on dev and production

✓ DevOps Solves These Issues

DevOps brings:

- Collaboration
- Automation
- Fast, reliable deployments
- Continuous feedback
- Shared responsibility

✓ DevOps Focus Areas

1. Planning

2. Coding
3. Building
4. Testing
5. Releasing
6. Deploying
7. Monitoring
8. Feedback

DevOps makes the entire software development life cycle continuous, forming a loop (infinity symbol).

3. When Did DevOps Come?

DevOps emerged because the industry was facing problems integrating development and operations.

✓ Timeline

2007–2008

- Companies saw major delays in deployments.
- Developers blamed operations, operations blamed developers.
- Patrick Debois, an IT consultant, was working on Agile Infrastructure, trying to combine development & operations.

2009

- Patrick held the first DevOpsDays conference in Belgium.
- The term “DevOps” became officially recognized.
- Many companies began adopting DevOps principles.

2010 onwards

- Major companies like Amazon, Netflix, Google transformed their pipelines using DevOps.

- Tools like Docker (2013) and Kubernetes (2014) accelerated DevOps growth.
-

4. Why DevOps Was Needed?

✓ Development Team Responsibilities

- Design UI/UX
- Write code
- Fix bugs
- Build the application
- Add new features

✓ Operations Team Responsibilities

- Manage servers
- Configure networks
- Deploy applications
- Monitoring
- Security
- Scaling & backup

✓ Major Problems Before DevOps

1. The Blame Game

Example:

- Developer pushes code to GitHub.
- Operations deploys it to the server.
- Application breaks.

Developers say → "Server issue"

Operations say → "Code issue"

2. Manual Deployments = Human Errors

- Steps done manually → mistakes happen.
- Configuration mismatches.
- Long downtime.

3. Slow Feedback

- Developers get bug reports after weeks or months.
- Fixing issues becomes more expensive and time-consuming.

4. Environment Mismatch

The famous line:

"It works on my machine!"

Developer's local environment - staging - production.

5. Slow Release Cycles

Companies released updates:

- Once every few months, or even annually.

DevOps changed this to:

- Multiple deployments per day (like Amazon, Netflix).

5. DevOps Toolchain

DevOps uses multiple tools, each for a specific purpose:

A. Source Code Management Tools

Git

- Tracks changes in code.
- Allows team collaboration.
- Supports branching & merging.

GitHub / GitLab / Bitbucket

- Cloud platforms built on Git.
 - Provide pull requests, code reviews, and repository hosting.
-

B. CI/CD Tools

CI/CD means automating build, test, and deployment processes.

Tools:

- Azure Pipelines
- GitHub Actions
- Jenkins
- GitLab CI
- CircleCI

These tools:

- Build the application
 - Run tests automatically
 - Deploy to servers with zero downtime
-

C. Containers

Docker

- Packages application + dependencies into a container.
 - Same environment everywhere.
 - "If it works in Docker, it works everywhere."
-

D. Container Orchestration

Kubernetes (K8s)

- Runs many containers.
 - Auto-scaling
 - Load balancing
 - Self-healing (restarts failed containers)
-

E. Monitoring Tools

- Azure Monitor
 - Grafana
 - Prometheus
 - Datadog
- These tools monitor:
- CPU usage
 - Crashes
 - Logs
 - Performance
 - Errors
-

6. Issue: Installing All Tools Separately

Each DevOps tool needed:

- Separate installation
- Separate login
- Separate dashboard
- Manual integrations
- Different monitoring panels

Companies needed one combined DevOps system.
This is where Azure DevOps helps.

7. What is Azure DevOps?

Azure DevOps is Microsoft's complete DevOps platform that integrates every stage of the DevOps lifecycle in a single system.

It provides:

- Code management
- Pipelines
- Testing
- Project tracking
- Package management

✓ Why Companies Use Azure DevOps?

- It's secure
 - Easy to set up
 - Works with any language
 - Integrates with Azure Cloud
 - Provides both GUI and YAML pipelines
 - Reduces DevOps complexity
-

8. Components of Azure DevOps

1. Azure Boards

Used for:

- Managing tasks
- User stories

- Sprints
- Scrum & Kanban boards

Developers and managers track all project progress here.

2. Azure Repos

Cloud-based Git repositories.

Features:

- Unlimited private repos
 - Pull requests
 - Branch policies
 - Code reviews
 - Merge checks
-

3. Azure Pipelines

The heart of Azure DevOps.

It automates:

- Building code
- Running tests
- Packaging
- Deployment

Supports:

- GitHub
- GitLab
- Bitbucket

- Azure Repos

Works with any language:

- Python
 - Java
 - .NET
 - Node.js
 - PHP
 - Go
-

4. Azure Test Plans

Used for:

- Manual testing
 - Automated testing
 - Capturing defects
 - Reporting test results
-

5. Azure Artifacts

Stores packages like:

- npm
- nuget
- Maven
- Python packages

Useful for large organizations needing shared libraries.

9. What is Microsoft Azure?

Azure is Microsoft's cloud computing platform with 200+ services.

✓ Azure Provides:

- Virtual Machines
- Databases
- AI Services
- Serverless functions
- Kubernetes cluster
- Storage accounts
- Networking firewalls

✓ Why Use Azure?

- Enterprise-grade security
 - Global data centers
 - Integration with DevOps
 - Cost-efficient
 - Auto-scaling
-

10. What Is a Resource Group?

A Resource Group (RG) is like a folder that stores all Azure resources.

Example:

If you create a web application, you may have:

- App Service
- SQL Database
- Storage Account
- Function App

All stored inside one Resource Group for:

- Organization
 - Billing
 - Easy deletion
 - Same region control
-

11. What Is a Function App?

A Function App is Azure's serverless compute service.

✓ What "Serverless" Means

- You do NOT manage servers.
- You pay only when your function runs.
- Azure handles scaling.

✓ Use Cases

- Sending emails
- Cron jobs
- Background processing
- Database automation
- Event-driven systems

✓ Important Azure Function Triggers

1. HTTP Trigger → Runs when an HTTP request comes.
2. Timer Trigger → Scheduled jobs like "*run every 5 minutes*".
3. Queue Trigger → Runs when a message arrives.
4. Blob Trigger → Runs when a file is uploaded.
5. Event Grid Trigger → Reacts to Azure events.

12. What Is CI/CD?

A. CI — Continuous Integration

Process where:

- Developer pushes code →
- Pipeline automatically:
 - Builds code
 - Runs unit tests
 - Checks errors
 - Validates quality

✓ Purpose:

- Catch bugs early
- Ensure code is always working
- Maintain stable builds

B. CD — Continuous Delivery / Deployment

After CI:

- Automatically deploys the build
- To staging or production

✓ Goal:

- Zero downtime
 - Fast releases
 - Automation of deployment
-

13. What Is a YAML Pipeline File?

A YAML file is a configuration file that defines:

- What should trigger the pipeline
- Which machine to run on
- What steps to execute
- How to deploy

✓ Detailed Example YAML

trigger:

- main # Pipeline runs whenever code is pushed to main branch

pool:

vmImage: 'ubuntu-latest' # Uses Ubuntu machine for running pipeline

steps:

- task: UsePythonVersion@0

inputs:

versionSpec: '3.10'

- script: pip install -r requirements.txt

displayName: Install dependencies

- script: python app.py

displayName: Run the application

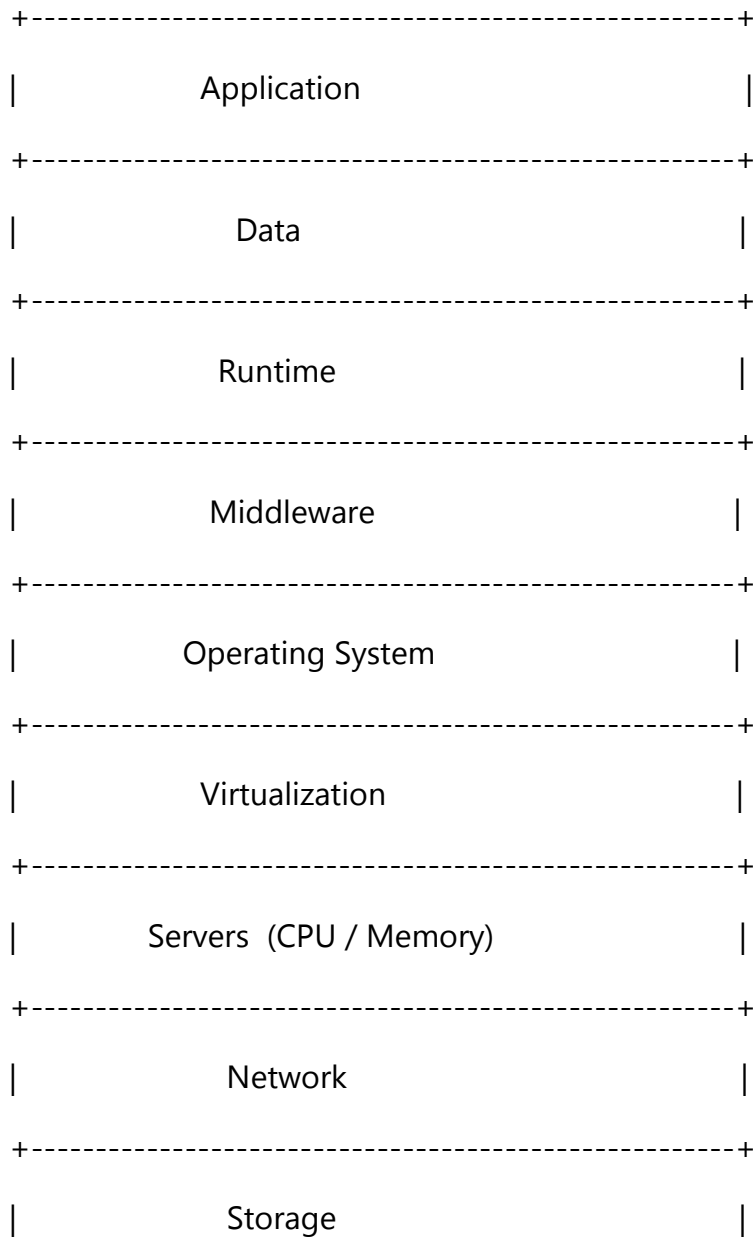
✓ Breakdown

- trigger: when pipeline should run
- pool: machine type
- steps: list of tasks
- script: commands to execute

YAML pipelines give:

- More control
- Reusable stages
- Version control for pipeline
-

LAYER DIAGRAM (PICTURE OF THE COMPLETE STACK)



+-----+

AZURE SERVICE MODELS

1. IAAS — Infrastructure as a Service (Most Control)

IaaS gives you raw computing resources such as:

- Virtual Machines
- Networking
- Storage

You manage the software layer; Azure manages the hardware layer.

Responsibilities in IaaS

YOU Manage:

(Everything above Operating System)

Application

Data

Runtime

Middleware

Operating System

MICROSOFT Manages:

(Everything below OS)

Virtualization

Servers (CPU/Memory)

Network

Storage

Example of IaaS

- Azure Virtual Machine (VM)
- You get a machine (like your laptop in cloud)
- You install OS updates, software, antivirus, frameworks

Example Scenario

A company needs a custom environment to host its backend:

- Install Python, Java, Node.js manually
 - Install custom databases
 - Full admin access to OS
-

When to Use IaaS

- You need full control over OS
 - Want to run custom apps
 - Want to migrate on-prem servers to cloud
 - Need your own security tools
-

2. PaaS — Platform as a Service (Balanced Control)

PaaS provides:

- Runtime environment
- Middleware
- Operating System
- Built-in scaling

Developers only focus on app + data.

Responsibilities in PaaS

✓ YOU Manage:

Application

Data

✓ MICROSOFT Manages:

Runtime

Middleware

Operating System

Virtualization

Servers

Network

Storage

Example of PaaS

- Azure App Service
- Azure SQL Database (managed)

Example Scenario

A development team wants to:

- Deploy a web app quickly
- Not worry about OS updates
- Scale automatically

Azure App Service handles all backend OS, runtime, autoscaling.

When to Use PaaS

- Fast development
 - No OS management
 - Automatic scaling
 - Cost-efficient
-

✅ 3. SAAS — Software as a Service (Least Control)

SaaS provides a ready-made application that users access through a browser.

You don't manage:

- Servers
 - Runtime
 - OS
 - Data layer (except limited settings)
-

📌 Responsibilities in SaaS

✅ YOU Manage:

Nothing major

(Only user preferences & basic settings)

✅ MICROSOFT Manages:

Application

Data

Runtime

Middleware

Operating System

Virtualization

Servers

Network

Storage

Examples of SaaS

- Microsoft 365 (Word, Excel, Outlook)
- GitHub
- Gmail
- Salesforce

Example Scenario

A company wants:

- Email service
 - Office apps
 - Cloud storage without managing anything.
-

When to Use SaaS

- Business tools
 - Zero maintenance
 - No IT team
 - Fast onboarding
-

4. FAAS — Function as a Service (Serverless)

FaaS is a part of Serverless. You write only the function code, Azure handles everything else.

This is used for:

- Event-driven applications
 - Background tasks
 - Microservices
-

Responsibilities in FaaS

✓ YOU Manage:

Function code (Application)

Data (optional)

✓ MICROSOFT Manages:

Runtime

Middleware

Operating System

Virtualization

Servers

Network

Storage

Scaling

Example of FaaS

- Azure Functions

Example Trigger Scenarios:

- A file is uploaded → function runs
- A timer runs every 5 minutes

- A queue receives a message
 - HTTP request trigger's function
-

When to Use FaaS

- Background jobs
 - On-demand execution
 - Pay only when function runs
 - No server management
-